Project Clara

Sprint 5

Foxcorp
Team Members:

- Abeer Syed
- Abraham Abebe
- Douglas Borovina
- Hayat Dahraj
- Jacob Gallagher

Problem Statement

- **Currently, Parents have no good way of knowing what their child is doing at school, which contributes to several problems in both the families' lives and the teachers'.**

- Parent anxiety about child's whereabouts throughout the day
- No information on child's wellbeing throughout the day
- Lack of parental engagement in child's education
- No efficient channels to communicate concerns to teachers
- Burden on teacher to manage several fragmented communication channels

Messaging System: High-Level Architecture

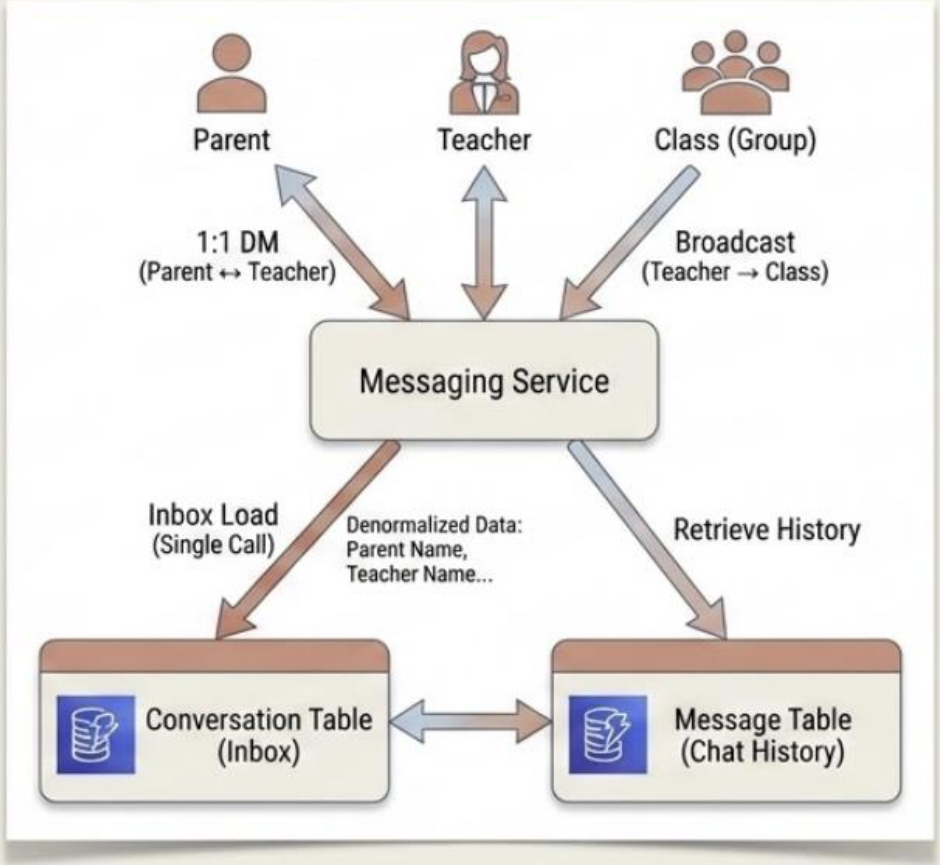
Core Capabilities: Supports 1:1 Direct Messages (Parent ↔ Teacher) and Group Broadcasts (Teacher → Class).

Optimized Database Structure: Split into two DynamoDB tables to maximize mobile performance.

Conversation Table: Acts as the "inbox".

Message Table: Stores the actual chat history.

Performance Win (Denormalization): Parent and Teacher names are stored directly on the **Conversation** row. This eliminates "N+1 query" bottlenecks, allowing the inbox to load instantly in a single database call.



Data Denormalization: Making the Inbox Load Instantly

Normalized (Two Separate Tables)

Conversation Table

id	teacherId	parentId	lastMessage
conv-1	teacher-abc	parent-xyz	See you Monday
conv-2	teacher-def	parent-xyz	Field trip Friday

Teacher Table

id	firstName	lastName
teacher-abc	Mrs.	Smith
teacher-def	Mr.	Jones

Requires a second query to find 'Mrs. Smith' using teacher-abc

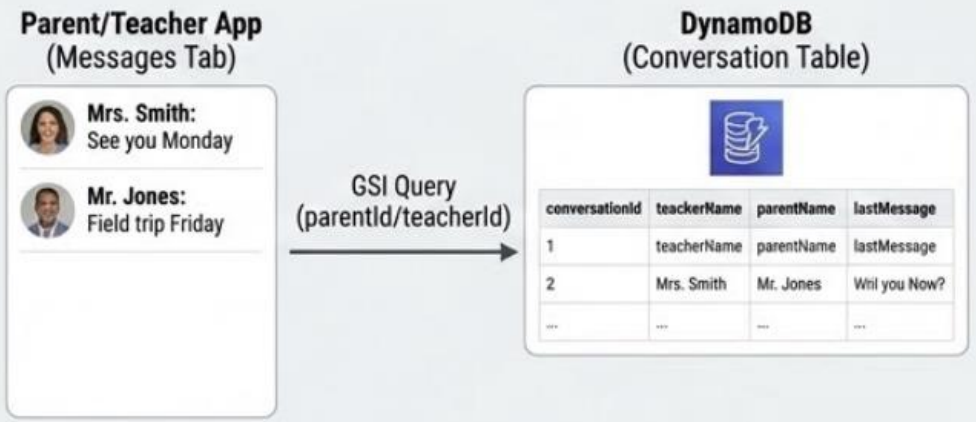
Denormalized (Data Copied to One Table)

Conversation Table

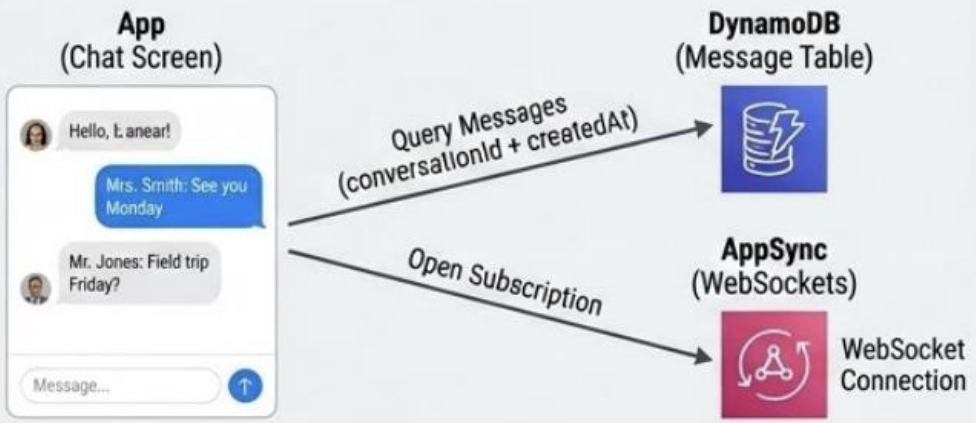
id	teacherId	**teacherName**	parentId	lastMessage
conv-1	teacher-abc	**Mrs. Smith**	parent-xyz	See you Monday
conv-2	teacher-def	**Mr. Jones**	parent-xyz	Field trip Friday

The name is already here! No second query needed. Single call to load inbox.

Denormalization is simply storing the answer ahead of time so you don't have to go find it later.

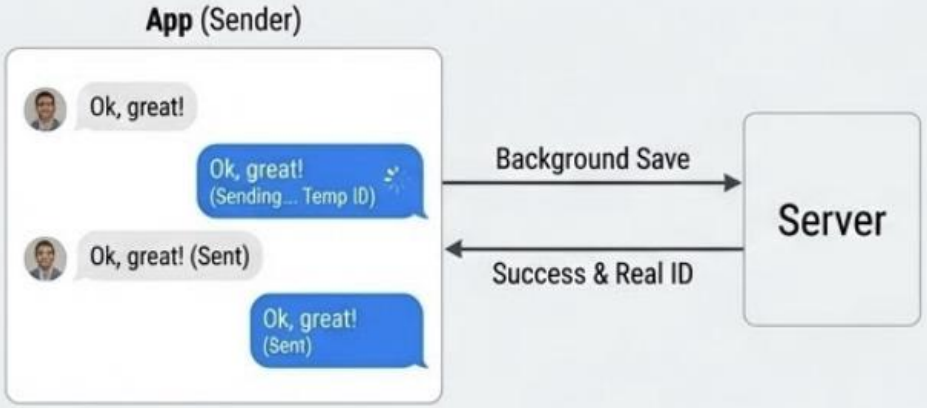


Denormalized data allows instant inbox rendering with a single query.



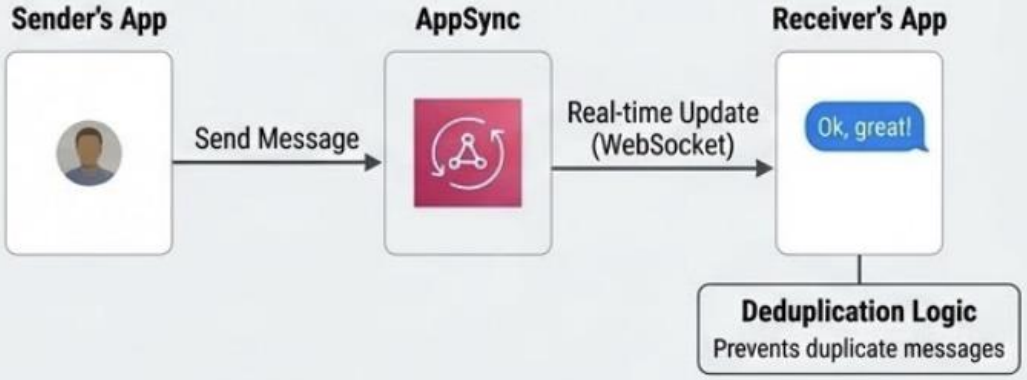
Simultaneous query for history and subscription for real-time updates.

Sending a Message (Optimistic UI)



Message appears instantly; real save and ID swap happen in background.

Receiving a Message (Real-time)



New message delivered via WebSocket in <300ms, with deduplication.

Optimized Database Design: Conversation & Message Tables

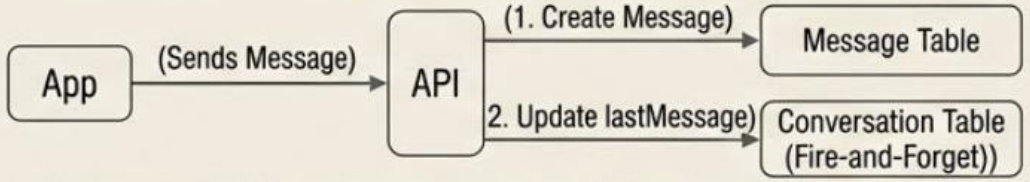
Conversation Table (Inbox View)

- **Stores Conversation Threads:** Holds metadata, not message history.
- **Denormalized Data:** Includes participant names (parentName, teacherName, etc.) to avoid joins.
- **Inbox Preview Fields:** Stores `lastMessageText` and `lastMessageAt` for instant rendering.
- **Performance:** Sorts inbox by recency and shows preview without querying the Message table.
- **Global Secondary Indexes (GSIs):** Four GSIs (e.g., byParent, byTeacher) allow fast, targeted queries (filtering at the database level) instead of slow table scans.

Message Table (Chat History)

- **Stores Individual Messages:** The actual chat content.
- **Composite Key Design:** Uses `conversationId` as Partition Key and `createdAt` as Sort Key.
- **Pre-Sorted Retrieval:** Querying "all messages for conversation X" returns them already sorted by time (oldest to newest).
- **Performance Win:** No application-level sorting required, significantly boosting mobile performance.

Efficient Write Operations (Sending a Message)



The conversation update is asynchronous; a slight inbox staleness is acceptable for faster message sending.

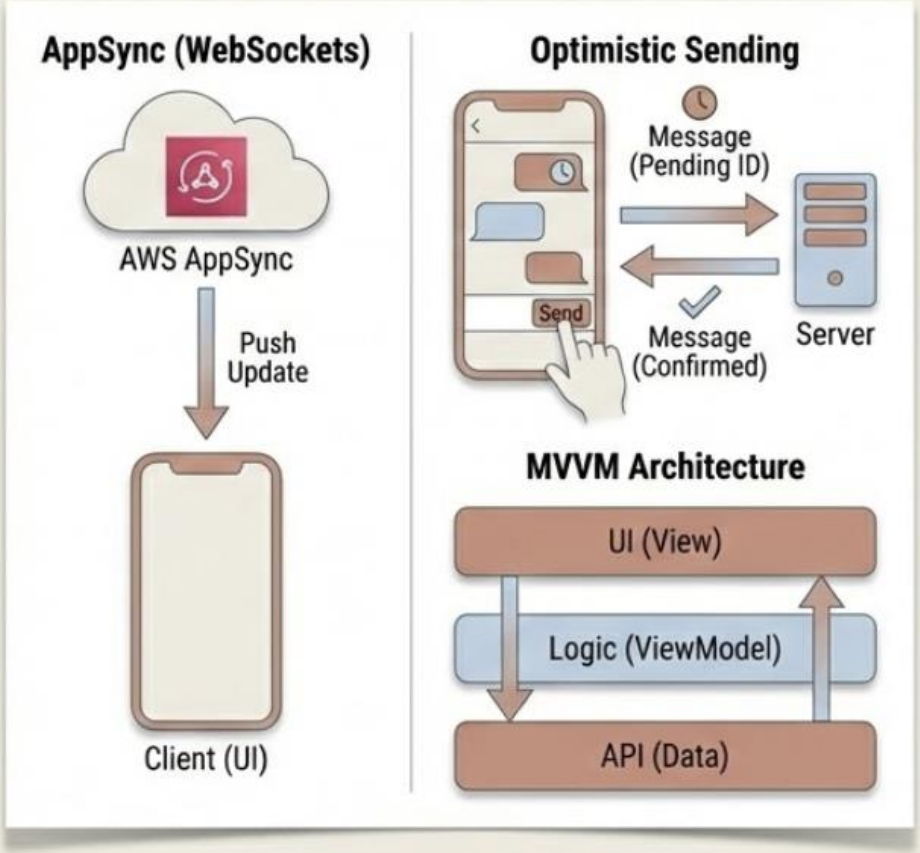
Real-Time Sync & Optimistic UI

AppSync Subscriptions (WebSockets): True real-time messaging. New messages are pushed to the user's screen instantly without needing to swipe-to-refresh or poll the server.

Optimistic Sending: When a user taps 'Send', the message appears in the chat UI instantly with a temporary ID. The server confirmation happens silently in the background.

Strict MVVM Architecture:

- **api/** (Data): messageRepo.ts is the single source of truth for AWS.
- **logic/** (ViewModel): Custom hooks handle states and deduplication.
- **ui/** (View): Visual components remain completely isolated from GraphQL.



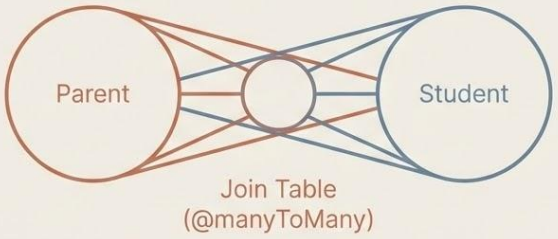
The Hand-off: What's Next?

1. **Unblock Teacher Screens (High Priority):**
   - Expand TeacherLoginContext to expose teacher.id and currentClass.id.
   - Swap out the placeholder constants in the teacher messaging files.
2. **Wire Up "Contact Teacher" Flow:**
   - Connect the parent's "Message Teacher" button to the existing getOrCreateDirectConversation() function.
3. **Apply Final Styling:**
   - UI components are built and already support Dark Mode via useThemeColor. Just need final padding/color tweaks to match Figma.

Modeling Complex Family Structures

What it means: Designed the database for real-world families, creating a complex link where one parent can have multiple children, and one child can have multiple parents.

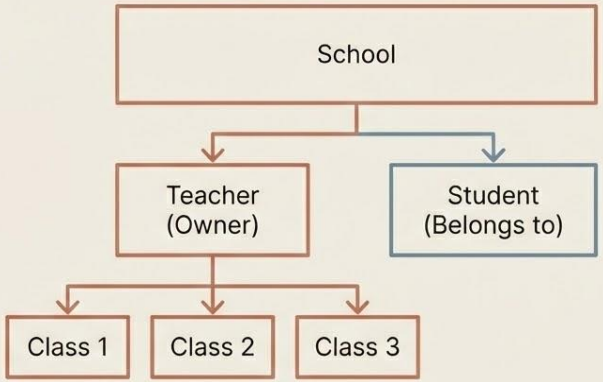
Why it matters: Prevents data duplication and allows for flexible family structures. The @manyToMany command automatically builds the required complex “join table”.



Establishing Data Hierarchy & Source of Truth

What it means: Established a strict hierarchy. A “Student” belongs to a specific School, and a “Teacher” is the owner of multiple Classes.

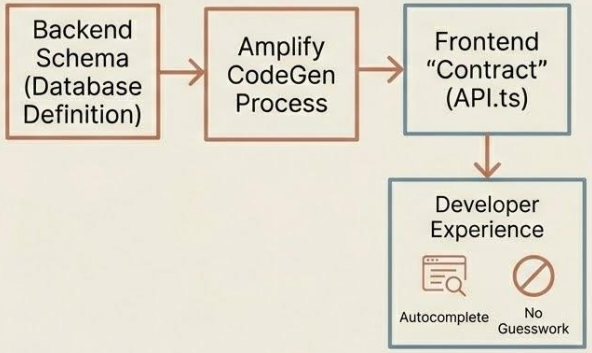
Why it matters: Creates a “Source of Truth”. Keeps data organized and logical, ensuring the database knows which which Students are affected if a School is deleted.



Accelerating Frontend Development with Auto-Generated Types

What it means: Set up the backend to write code for the frontend team automatically. The defined database structure (Schema) auto-generates the ‘Contract’ (API.ts) that strictly defines data.

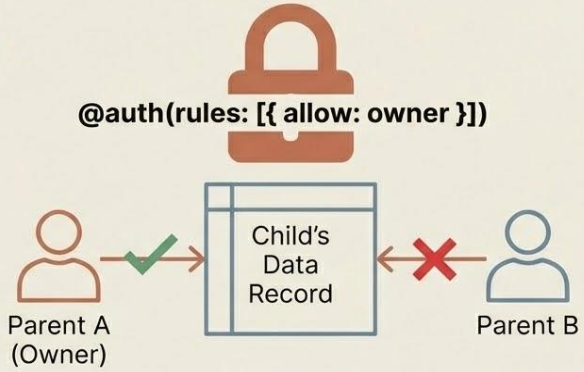
Why it matters: Unblocks the team. Frontend developers import auto-generated code, eliminating guesswork and enabling features like autocomplete immediately.



Enforcing Record-Level Data Security

What it means: Implemented a “Digital Lock” on every record. The @auth rule creates a strict policy: “If you didn’t create this data, you can’t see it.”

Why it matters: Guarantees privacy by making it physically impossible for one parent to see another’s children, meeting security standards without custom code.



Backend – 1
Backend Data Architecture

Backend Engineering (AWS & GraphQL)

Challenge: App relied on hardcoded JSON; needed a real relational database.

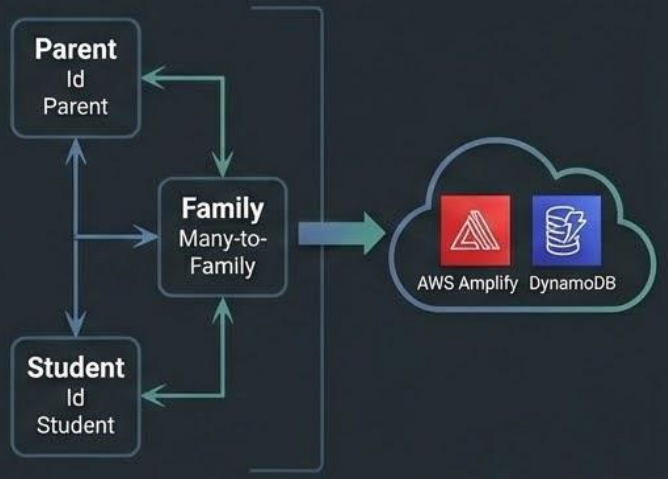
Solution: Designed and deployed a GraphQL Schema via AWS Amplify.

Key Technicals:

Modeled `Parent` and `Student` entities.

Implemented Many-to-Many relationships (@manyToMany` directive) for family mapping.

Enforced security via `@auth(rules: [{allow: owner}})`.



Database: Tables Created

Database Tables

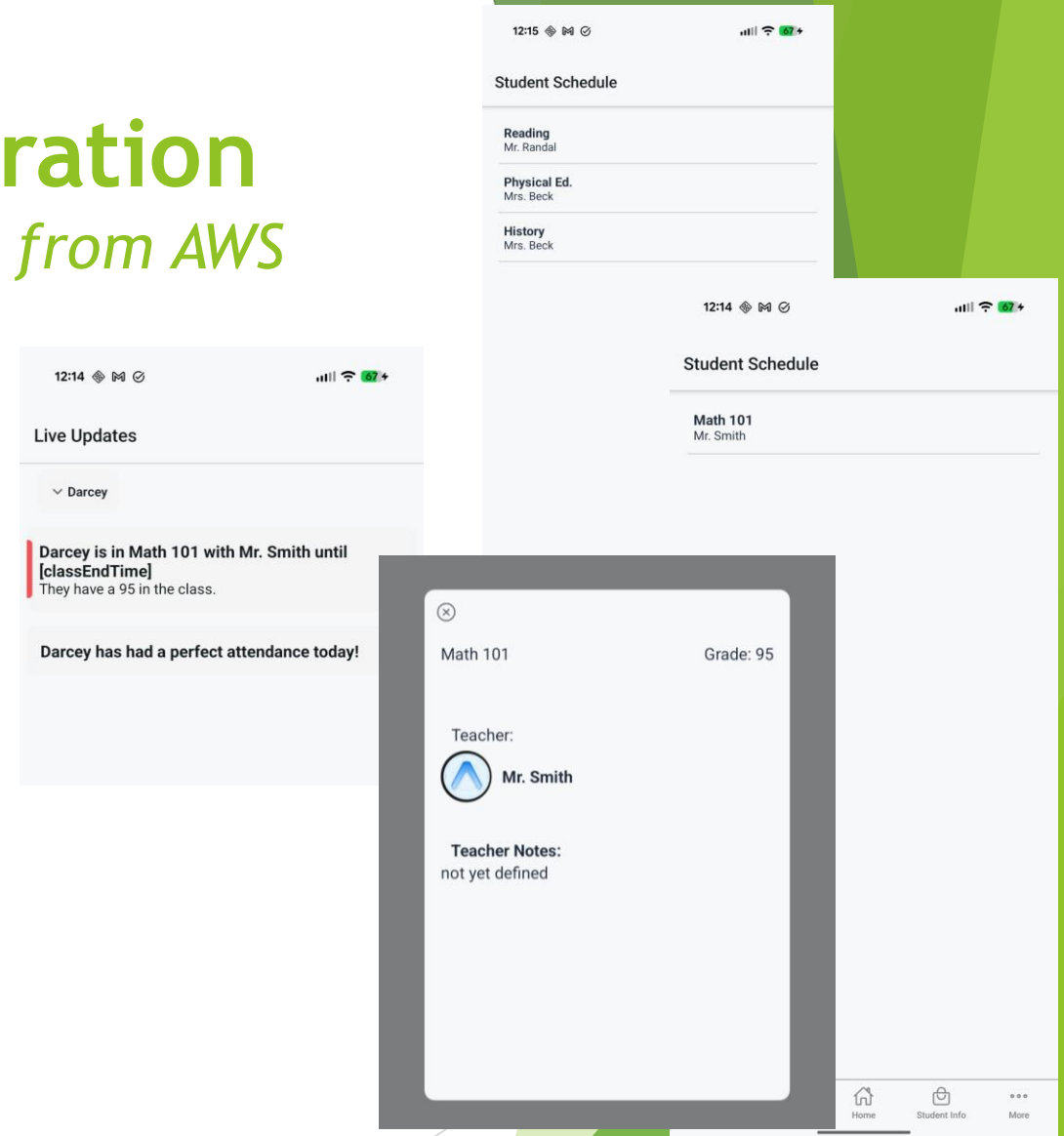
Table	Key Fields	Purpose
School	name, address	School selection on login
Parent	cognitoUserId, firstName, lastName	Parent accounts linked to Cognito login
Student	firstName, lastName, gradeLevel, attendanceRate, currentStatus	Student profiles
ParentStudents	parentId, studentId	Join table linking parents to kids
Teacher	cognitoUserId, name, schoolId	Teacher accounts
Class	name, teacherId, schoolId	Classes per teacher per school
Enrollment	studentId, classId	Links students to their classes

Parent's Views Backend Integration

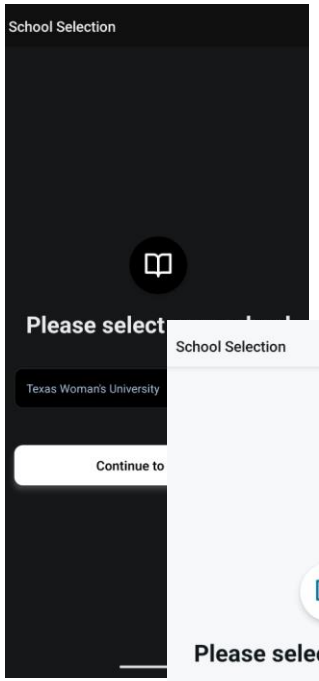
Grabbing Teacher, Class, and Enrollment info from AWS

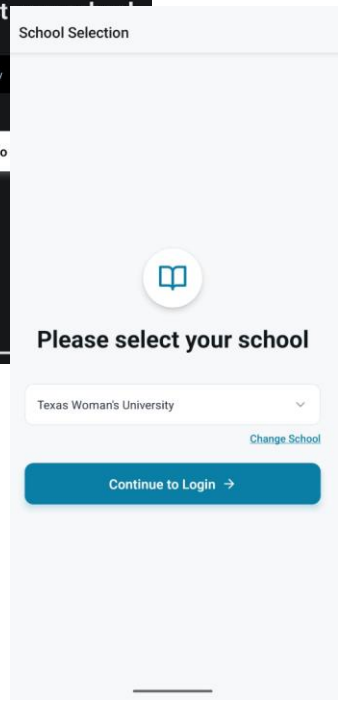
- In an expansion of last sprint, the Parent Views now query more info
- We now pull Class, Enrollment, and Teacher information

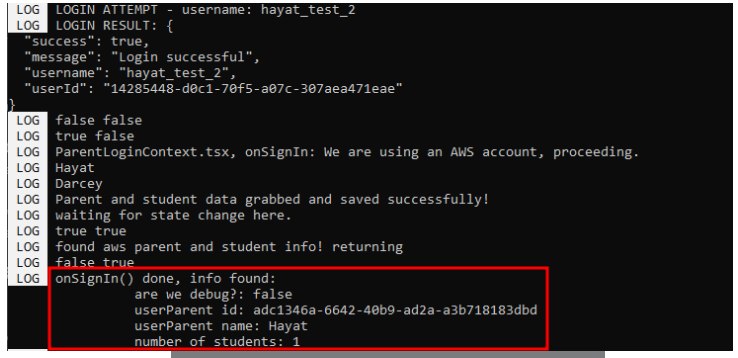
- Like Student and Parent data, this data is stored in a Context layer above the Parent views
- This data is accessible from anywhere after it has been loaded

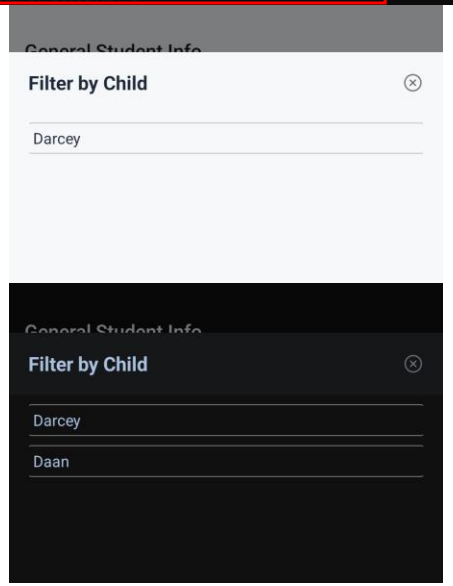


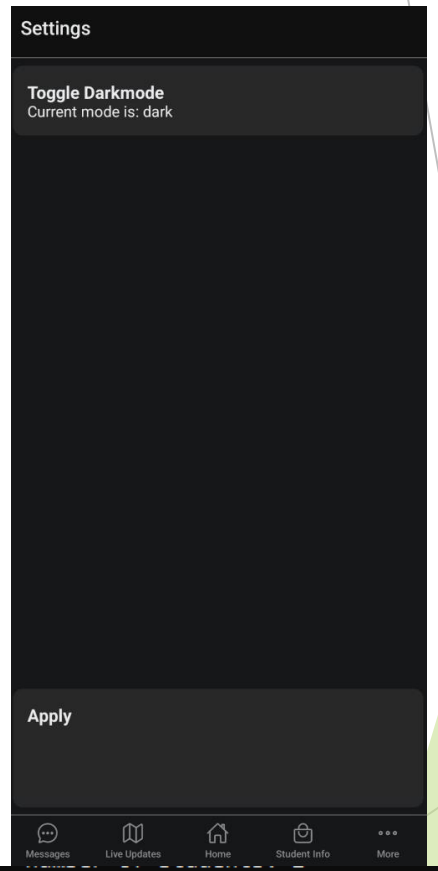
Briefly Showing What We Couldn’t Last Time

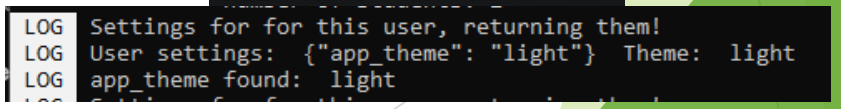






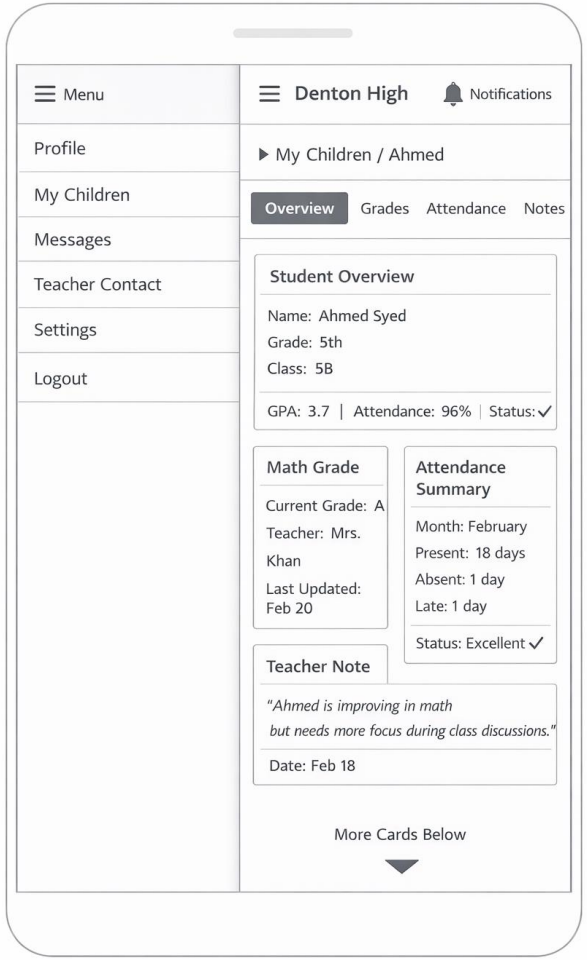


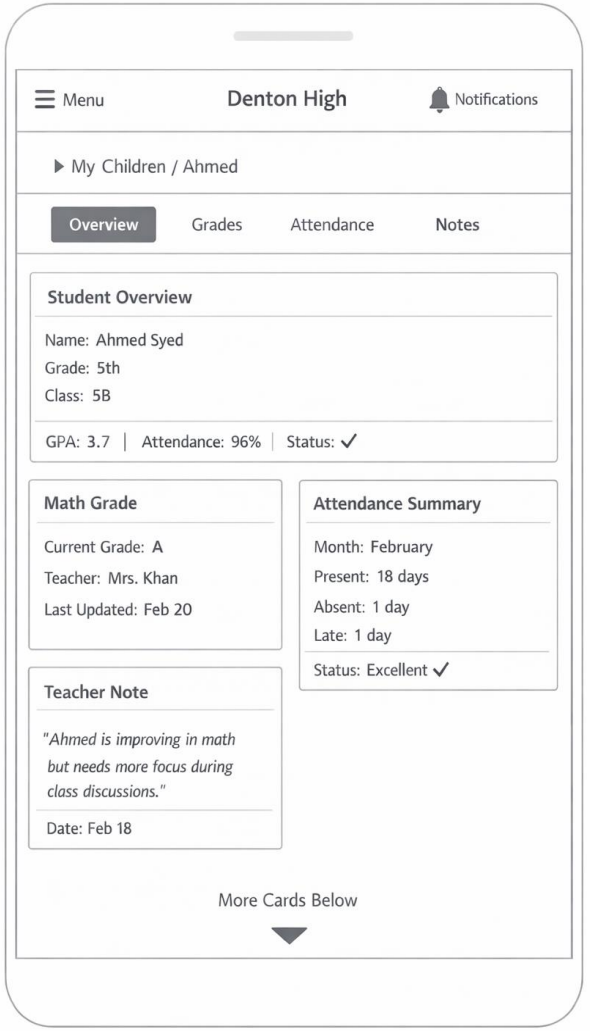




Proposed System Interface and Wireframe Design

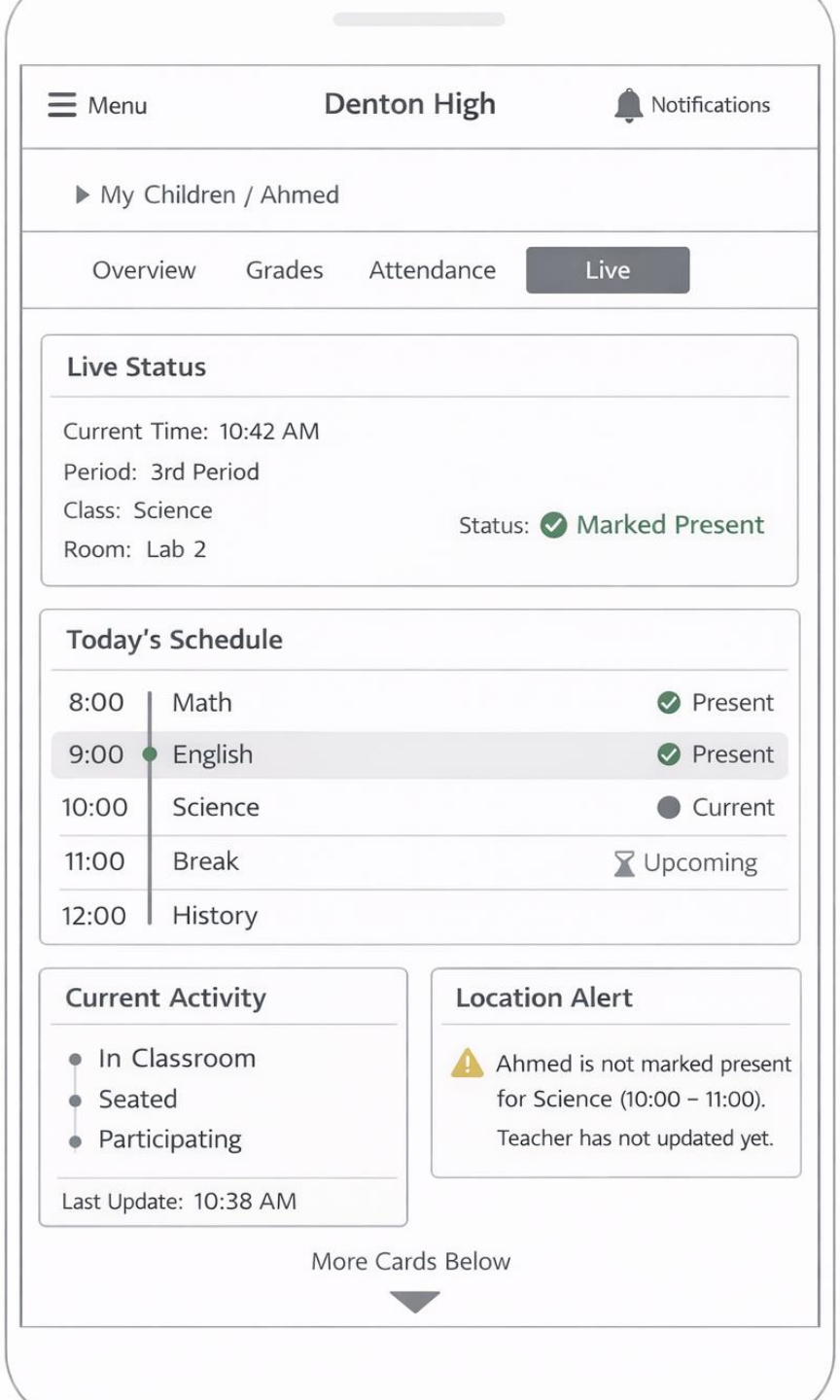
- **Profile:** Displays and allows editing of the parent's personal account information.
- **Children:** Shows a list of the parent's registered children with their academic details.
- **Messages:** Enables real-time communication between parents and teachers.
- **Teacher Contact:** Provides access to teacher information and direct contact options.
- **Settings:** Allows the parent to manage app preferences and account options.





Design and Wireframe for Student Location Tracking

- **Live Status (if included):** Displays the child's current class, attendance status, and schoolactivity in real time.
- Primarily will be based on event-based status instead of live GPS updates
- Teacher updates the attendance and based on time stamps the process will start and terminate
- Parents can view child current status
- Shows attendance status (Present / Not Marked / Absent)
- Highlights current class within the full daily schedule



Test Cases

TEST CASE ID	DESCRIPTION	REQUIREMENT TRACE	DIRECTIONS	EXPECTED OUTPUT	PASS/FAIL
TCB-007	Verify correct account data fetched and used by Frontend from AWS	R31, R32, R36	1. Login with a Parent Cognito account 2. Go to “General Info” screen 3. Tap the Student Picker dropdown 4. Check Class and Teacher names based on which Student is selected 5. Select a student 6. Verify information in screen reflects Student information accurately 7. Repeat steps 5-6 for all Classes and Teachers available	The Classes and Teachers connected to the parent’s account should all be loaded in and shown successfully. Each Class and Teacher should have accurate information provided by the backend.	Pass
TCB-701	Verify app appearance global variable reflects OS theme by default	R16	1. Wipe app cache, ensuring app has no saved settings 2. Change OS theme to “light” mode 3. Open app to login screen 4. Observe color theme 5. Repeat steps 1-4 with “dark” mode OS theme	The app will reflect the color scheme of the OS’s ui. Console will log ui as “dark” or “light”, matching the scheme of the OS.	Pass
TCB-702	App appearance changes when changing UI theme in settings	R16	1. Login to the app and navigate to the “Settings” screen 2. Tap the UI theme toggle 3. Hit “Apply” to apply the settings.	The app will change UI colors to reflect the newly chosen theme instantly. Console will log both the changing of theme aswell as saving the new settings to local storage. Both should be logged with no errors.	Pass

Improvements to Documents

Design Document:

- Updated all Screenshots to reflect changes

Test Document:

- Added black-box test cases for testing AWS Graphql Classes, Teachers and Enrollments data

Before the Sprint is Over

- Write up Sprint Submission Report
- Finish test cases
- Finish implementing changes in Teacher side
- Update test plan to reflect new changes

Focus of Next Sprint

- Expanding the AWS backend to include more data for Announcements, StudentDocumentation, and Events
- Fully implement the new Messaging features